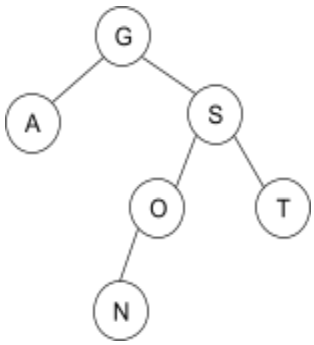
American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Intermediate Division

PROBLEM: The *Binary Search Tree* built from the letters GASTON is shown on the right. One way to represent this data structure is to use 3 arrays:



- an array of the *keys* in the order that they appear in the string
- an array of the position of each key's *left* child,
- an array of the position of each key's *right* child.

To add a new key, find where it must go in the tree and update the value of the parent's left or right array with the key's location. A value of -1 in the left or right array indicates that the node does not have that child. Here are the 3 arrays that are created from the letters GASTON:

	0	1	2	3	4	5	…
keys	**G**	**A**	**S**	**T**	**O**	**N**	
left	1	-1	4	-1	5	-1	
right	2	-1	3	-1	-1	-1	

In this program, you will be given a string to create a binary search tree from its letters. Once the tree is built, you will be given a letter to add to the tree. Output the path of subscripts that is followed to add a new letter. Follow that by an L or an R to indicate whether it becomes the left or right child of its parent.

EXAMPLE:

Input	`GASTON P`
Output	`0 2 4 R`
Explanation	Initialize all elements in the left and right arrays to -1. The letter **G**: set key[0]='G'. The letter **A**: set key[1]='A'. A is the left child of G, so left[0]=1. The letter **S**: set key[2]='S'. S is the right child of G, so right[0]=2. The letter **T**: set key[3]='T'. T is the right child of S, so right[2]=3. The letter **O**: set key[4]='O'. O is the left child of S, so left[2]=4. The letter **N**: set key[5]='N'. N is the left child of O, so left[4]=5. To insert the letter **P**: It is greater than **G** in location 0, less than or equal to **S** in location 2, and greater than **O** in location 4. The key **O** doesn't have a right child, so the **P** would become the right child of the **O**. The output is `0 2 4 R`.

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Intermediate Division

INPUT: Input two strings that are all uppercase alphabetic characters. The first string is used to build the binary search tree. The second is a single character to be added to the tree. The first string will be no more than 20 characters long.

OUTPUT: Output a string of numeric values, each separated by a single space, representing the position in the array of the keys that are traversed when adding the letter in the second input string. Follow that with the letter L or R, indicating if the added letter is a left or right child of its parent.

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Intermediate Division

SAMPLE INPUT:

```
1. GASTON
   P
2. JASMINE
   P
3. PETERPAN
   K
4. RAPUNZEL
   S
5. PINOCCHIO
   D
6. MICKEYMOUSE
   L
```

SAMPLE OUTPUT:

```
1. 0 2 4 R
2. 0 2 3 5 R
3. 0 1 5 7 L
4. 0 3 L
5. 0 1 4 6 L
6. 0 1 3 6 L
```

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Intermediate Division

TEST DATA

TEST INPUT:

```
1. CINDERELLA
   M
2. MALEFICENT
   G
3. DONALDDUCK
   M
4. BUZZLIGHTYEAR
   W
5. SLEEPINGBEAUTY
   H
6. A
   A
```

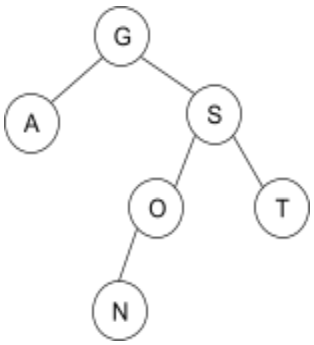
TEST OUTPUT:

```
1. 0 1 2 7 R
2. 0 1 2 3 4 5 L
3. 0 1 2 4 R
4. 0 1 2 3 9 L
5. 0 1 2 5 7 R
6. 0 L
```

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Intermediate Division

PROBLEM STATEMENT:



The *Binary Search Tree* built from the letters GASTON is shown on the right. One way to represent this data structure is to use 3 arrays:

- an array of the *keys* in the order that they appear in the string
- an array of the position of each key's *left* child,
- an array of the position of each key's *right* child.

To add a new key, find where it must go in the tree and update the value of the parent's left or right array with the key's location. A value of -1 in the left or right array indicates that the node does not have that child. Here are the 3 arrays that are created from the letters GASTON:

	0	1	2	3	4	5	…
keys	**G**	**A**	**S**	**T**	**O**	**N**	
left	1	-1	4	-1	5	-1	
right	2	-1	3	-1	-1	-1	

In this program, you will be given a string to create a binary search tree from its letters. Once the tree is built, you will be given a letter to add to the tree. Output the path of subscripts that is followed to add a new letter. Follow that by an L or an R to indicate whether it becomes the left or right child of its parent.

EXAMPLE:

Input:

```
GASTON
P
```

Output:

```
0 2 4 R
```

Explanation:

Initialize all elements in the left and right arrays to -1.

The letter **G**: set key[0]='G'.
The letter **A**: set key[1]='A'. A is the left child of G, so left[0]=1.
The letter **S**: set key[2]='S'. S is the right child of G, so right[0]=2.
The letter **T**: set key[3]='T'. T is the right child of S, so right[2]=3.
The letter **O**: set key[4]='O'. O is the left child of S, so left[2]=4.
The letter **N**: set key[5]='N'. N is the left child of O, so left[4]=5.

To insert the letter **P**:
It is greater than **G** in location 0, less than or equal to **S** in location 2, and greater than **O** in location 4. The key **O** doesn't have a right child, so the **P** would become the right child of the **O**. The output is `0 2 4 R`.

TASK:

Complete the function **growTree**

- The function has 2 parameters: a string of uppercase letters, *word*, that is used to represent a binary search tree in 3 one-dimensional arrays and a one-character string, *letter*, for the letter to be inserted into the existing tree once it is built.
- The function returns a string of numeric values representing the subscripts of the letters referenced when adding that letter to the existing tree followed by the letter L or R, indicating if the added letter is a left or right child of its parent. All output values are separated by a single space.

You may create additional functions that are called from **growTree** if needed in solving the problem.

CONSTRAINTS:

Each string will be no more than 20 characters. Use the ACSL definition for building the tree.

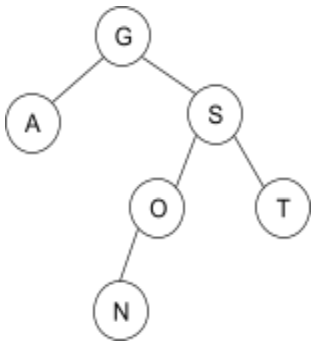
DATA PROVIDED:

There are 6 sets of Sample Data for debugging and 6 sets of Test Data for scoring. The test cases will vary in difficulty. You should create sample data of your own to fully test your program.

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Intermediate Division

问题：由字符串 GASTON 构建的*二叉搜索树*如右图所示。该数据结构可通过以下 3 个数组表示：



- 一个按字符串中出现顺序存储*键*的数组
- 一个数组记录每个键*左*子节点的位置
- 一个数组记录每个键*右*子节点的位置

插入新键时，需确定其在树中的位置，并将其父节点左子数组或右子数组的值更新为新键的位置。若左子数组或右子数组中的值为 -1，则表示该节点对应的子节点不存在。

以下是由字符串 GASTON 生成的三个数组：

	0	1	2	3	4	5	…
键	**G**	**A**	**S**	**T**	**O**	**N**	
左子节点	1	-1	4	-1	5	-1	
右子节点	2	-1	3	-1	-1	-1	

本程序将提供一个字符串，你需要根据其字母来构建二叉搜索树。树构建完成后，将提供一个字母添加到树中。请输出添加新字母时所经过的下标路径，并用 L 或 R 标明该字母是成为其父节点的左子节点还是右子节点。

示例：

输入	GASTON P
输出	0 2 4 R
解析	将左子节点数组和右子节点数组的所有元素初始化为 -1。 字母 **G**：设置 key[0]='G'。 字母 **A**：设置 key[1]='A'。A 是 G 的左子节点,故 left[0]=1。 字母 **S**：设置 key[2]='S'。S 是 G 的右子节点,故 right[0]=2。 字母 **T**：设置 key[3]='T'。T 是 S 的右子节点,故 right[2]=3。 字母 **O**：设置 key[4]='O'。O 是 S 的左子节点,故 left[2]=4。 字母 **N**：设置 key[5]='N'。N 是 O 的左子节点,故 left[4]=5。 插入字母 **P**： **P** 大于处于位置 0 的 **G** ，小于或等于处于位置 2 的 **S** ，且大于处于位置 4 的 **O**。键 **O** 没有右子节点，因此 **P** 将成为 **O** 的右子节点。输出结果为 0 2 4 R 。

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Intermediate Division

输入：输入两个全由大写字母组成的字符串。第一个字符串用于构建二叉搜索树，第二个是需添加到树中的单字符。第一个字符串长度不超过 20 个字符。

输出：输出一串用单个空格分隔的数值，表示添加第二个输入字符串中的字母时所遍历的键在数组中的位置。最后加上字母 L 或 R ，表明所添加字母是其父节点的左子节点或是右子节点。

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Intermediate Division

样例输入：

```
1. GASTON
   P
2. JASMINE
   P
3. PETERPAN
   K
4. RAPUNZEL
   S
5. PINOCCHIO
   D
6. MICKEYMOUSE
   L
```

样例输出：

```
1. 0 2 4 R
2. 0 2 3 5 R
3. 0 1 5 7 L
4. 0 3 L
5. 0 1 4 6 L
6. 0 1 3 6 L
```

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Intermediate Division

测试数据

测试输入：

```
1. CINDERELLA
   M
2. MALEFICENT
   G
3. DONALDDUCK
   M
4. BUZZLIGHTYEAR
   W
5. SLEEPINGBEAUTY
   H
6. A
   A
```

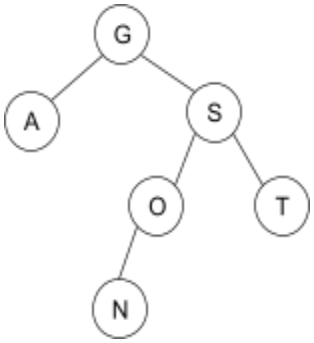
测试输出：

```
1. 0 1 2 7 R
2. 0 1 2 3 4 5 L
3. 0 1 2 4 R
4. 0 1 2 3 9 L
5. 0 1 2 5 7 R
6. 0 L
```

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Intermediate Division

问题陈述:



由字符串 GASTON 构建的*二叉搜索树*如右图所示。该数据结构可通过以下三个数组表示：

- 一个按字符串中出现顺序存储*键*的数组
- 一个数组记录每个键*左*子节点的位置
- 一个数组记录每个键*右*子节点的位置

插入新键时，需确定其在树中的位置，并将其父节点左子数组或右子数组的值更新为新键的位置。若左子数组或右子数组中的值为 -1，则表示该节点对应的子节点不存在。

以下是由字符串 GASTON 生成的三个数组：

	0	1	2	3	4	5	…
键	**G**	**A**	**S**	**T**	**O**	**N**	
左子节点	1	-1	4	-1	5	-1	
右子节点	2	-1	3	-1	-1	-1	

本程序将提供一个字符串，你需要根据其字母来构建二叉搜索树。树构建完成后，将提供一个字母添加到树中。请输出添加新字母时所经过的下标路径，并用 L 或 R 标明该字母是成为其父节点的左子节点还是右子节点。

示例：

输入：

```
GASTON
P
```

输出：

```
0 2 4 R
```

解析：

将左子节点数组和右子节点数组的所有元素初始化为 -1 。

字母 **G**：设置 key[0]='G'。
字母 **A**：设置 key[1]='A'。A 是 G 的左子节点,故 left[0]=1。
字母 **S**：设置 key[2]='S'。S 是 G 的右子节点,故 right[0]=2。
字母 **T**：设置 key[3]='T'。T 是 S 的右子节点,故 right[2]=3。
字母 **O**：设置 key[4]='O'。O 是 S 的左子节点,故 left[2]=4。
字母 **N**：设置 key[5]='N'。N 是 O 的左子节点,故 left[4]=5。

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Intermediate Division

插入字母 **P** ：
P 大于处于位置 0 的 **G** ，小于或等于处于位置 2 的 **S** ，且大于处于位置 4 的 **O**。键 **O** 没有右子节点，因此 **P** 将成为 **O** 的右子节点。输出结果为 `0 2 4 R` 。

任务:
完成函数 **growTree**

- 该函数有 2 个参数：一个由大写字母组成的字符串, *word*，用于通过 3 个一维数组表示二叉搜索树；以及一个单字符的字符串，*letter*，表示在树构建完成后要插入的字母。
- 函数返回一个由数值组成的字符串，表示向树中添加新字母时路径中经过的字母下标，并用 L 或 R 标明该字母是成为其父节点的左子节点还是右子节点。所有输出值用单个空格分隔。

如有需要，你可以构建额外的函数供 **growTree** 调用来解答此问题。

约束条件：
每个输入字符串长度不超过 20 个字符，使用 ACSL 定义的规则构建二叉树。

数据说明：
包含 6 组用于调试的样例数据和 6 组用于评分的测试数据。测试案例难度各不相同。您应该自行创建样例数据以充分验证程序。